NN

GenAI Engineer
Technical Assessment

A full-stack GenAI pipeline covering API integration, data transformation, testing, LLM identification, and agentic workflows — built with Python, Ollama, LangChain & LangGraph.

5 Exercises | Python + LangChain | ~90 Minutes | 10 Tests Passing

PREREQUISITES

Prerequisites

All dependencies verified ✓

cURL / Postman	Python 3.12	Ollama + llama3.2	LangChain
HTTP request tooling	Core runtime	Local LLM runtime	Agentic AI framework
● Ready	● Ready	● Ready	● Ready

pip3 install

`requests` `ollama` `pytest` `langchain` `langchain-ollama` `langchain-community` `langgraph` `duckduckgo-search` `tenacity` `python-dotenv`

```
$ pip3 install -r requirements.txt
```

EXERCISE 1 — API DATA FETCHING

API data fetching — Foundation

Fetch 20 random users from randomuser.me using Python's `requests` library with full error handling.

Library	Method	Format	Results
requests	GET	JSON	20 users

helperFunction.py — PYTHON

```python
import requests
from requests.exceptions import HTTPError, RequestException, JSONDecodeError
from datetime import datetime
```

```python

def get_users(results: int = 20) -> list:
    """Fetches random user data from randomuser.me API."""
    if results < 1 or results > 5000:
        raise ValueError("results must be between 1 and 5000")
    url = f"https://randomuser.me/api/?results={results}"
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()  # raises on 4xx / 5xx
        return response.json()["results"]
    except HTTPError as e:
        print(f"HTTP error: {e}"); raise
    except (JSONDecodeError, KeyError, RequestException):
        raise
```

SAMPLE CONSOLE OUTPUT

```
{
  "results": [
    { "name": { "first": "James", "last": "Wilson" }, "dob": { "age": 39 }, "nat": "GB" }
    // ... 19 more users
  ]
}
✅ Exercise 1 - Fetched 20 users
```

EXERCISE 2 — DATA TRANSFORMATION & FILTERING

🔍 Data transformation & filtering

Data processing

Format API response into `"First Last"` strings, filtering out anyone born after year 2000.

API response → Parse DOB → Filter > 2000 → Format name → Return list

helperFunction.py — PYTHON

```python
def format_users(users: list) -> list[str]:
    """Returns 'First Last' strings for users born ≤ 2000."""
    result = []
    for user in users:
        dob_str = user["dob"]["date"]
        birth_year = datetime.fromisoformat(
            dob_str.replace("Z", "+00:00")
        ).year
        if birth_year > 2000: # filter post-2000
            continue
        first = user["name"]["first"]
        last  = user["name"]["last"]
```

```
            result.append(f"{first} {last}")
        return result
```

CONSOLE OUTPUT (EXAMPLE WITH 20 USERS)

```
['Marie Lambert', 'Noah Hoffmann', 'Sofia Marchetti', 'James Wilson', 'Anna Kowalski',
'Carlos Romero', 'Elena Petrov', 'Hans Mueller', "Liam O'Connor"]
📌 15 of 20 users born in or before 2000 (5 filtered out)
```

EXERCISE 3 — UNIT & INTEGRATION TESTING

🧪 Unit & integration testing

Quality assurance

10 pytest tests — 9 unit + 1 integration — covering happy paths, edge cases, and mocked HTTP.

Total tests	Framework	Unit tests	Integration
10	pytest	9	1

- ✓ `test_returns_list` — Return type is always a list — Type safety
- ✓ `test_correct_count` — Correct number of filtered users — Filtering
- ✓ `test_name_format` — Names in "First Last" format — Formatting
- ✓ `test_filters_post_2000` — Post-2000 users excluded — Filtering
- ✓ `test_year_2000_edge_case` — Year 2000 births included — Edge case
- ✓ `test_empty_response` — Empty input returns empty list — Edge case
- ✓ `test_all_post_2000` — All post-2000 → empty result — Edge case
- ✓ `test_names_are_strings` — Names are non-empty strings — Type safety
- ✓ `test_missing_results_key` — Raises KeyError on bad API — Robustness
- ✓ `test_full_pipeline` — Mocked HTTP → end-to-end — Integration

PYTEST OUTPUT — ALL TESTS PASSING

```
platform darwin -- Python 3.12.8, pytest-9.0.3
collected 10 items
test_main.py::test_returns_list PASSED [ 10%]
test_main.py::test_correct_count PASSED [ 20%]
test_main.py::test_name_format PASSED [ 30%]
test_main.py::test_filters_post_2000 PASSED [ 40%]
test_main.py::test_includes_year_2000 PASSED [ 50%]
test_main.py::test_empty_response PASSED [ 60%]
test_main.py::test_all_post_2000 PASSED [ 70%]
test_main.py::test_names_are_strings PASSED [ 80%]
test_main.py::test_missing_results_key PASSED [ 90%]
test_main.py::test_full_pipeline PASSED [100%]
```

```
======================== 10 passed in 0.94s ========================
```

EXERCISE 4 — LLM PERSON IDENTIFICATION

🧠 LLM person identification

AI integration

Call Ollama / llama3.2 locally to identify whether each random user is a public figure.

Model	Temperature	Max tokens	Sample size
llama3.2	0.1	150	5 users

llm.py — PYTHON

```python
SYSTEM_PROMPT = """You are a strict fact-checker:
1. Search for the EXACT name given to you.
2. If results show a DIFFERENT person → NOT_NOTABLE.
3. Never substitute or guess a similar name."""

def identify_person(name: str, nationality: str, year: int) -> str:
    response = ollama.chat(
        model="llama3.2",
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user",   "content": prompt},
        ],
        options={"temperature": 0.1, "num_predict": 150}
    )
    return response["message"]["content"].strip()
```

CONSOLE OUTPUT

```
👤 Marie Lambert (FR, b. 1978)
   NOT_NOTABLE — private individual, no public records found.
👤 James Wilson (GB, b. 1985)
   NOT_NOTABLE — private individual, no public records found.
👤 Carlos Romero (ES, b. 1971)
   NOT_NOTABLE — private individual, no public records found.
```

Hallucination note: llama3.2 (3B) tends to substitute a well-known person for unknown names. Mitigations: `NOT_NOTABLE` sentinel + name-match check + `temperature=0.1`. For production, `gpt-4o-mini` is strongly recommended.

EXERCISE 5 — AGENTIC LANGCHAIN WORKFLOW

⚡ Agentic LangChain workflow Advanced AI

A LangGraph ReAct agent with DuckDuckGo search finds each person's best-known work. A name-match guard rejects wrong-person substitutions.

Task input → LLM reasoning → DuckDuckGo → Observation → Final answer

agent.py PYTHON

```python
from langchain_ollama import ChatOllama
from langchain_community.tools import DuckDuckGoSearchRun
from langgraph.prebuilt import create_react_agent

agent_executor = create_react_agent(
    model=llm, tools=[search_tool], prompt=SYSTEM_PROMPT
)

def get_best_work(name: str) -> str:
    result = agent_executor.invoke({"messages": [
        ("human", f"Who is {name} and what is their most acclaimed work?")
    ]})
    output = result["messages"][-1].content
    if "NOT_NOTABLE" in output: return "NOT_NOTABLE"
    if not names_match(name, output): return "NOT_NOTABLE"
    return output
```

BONUS — ENHANCEMENTS & BEST PRACTICES

⭐ Bonus: enhancements & best practices Bonus

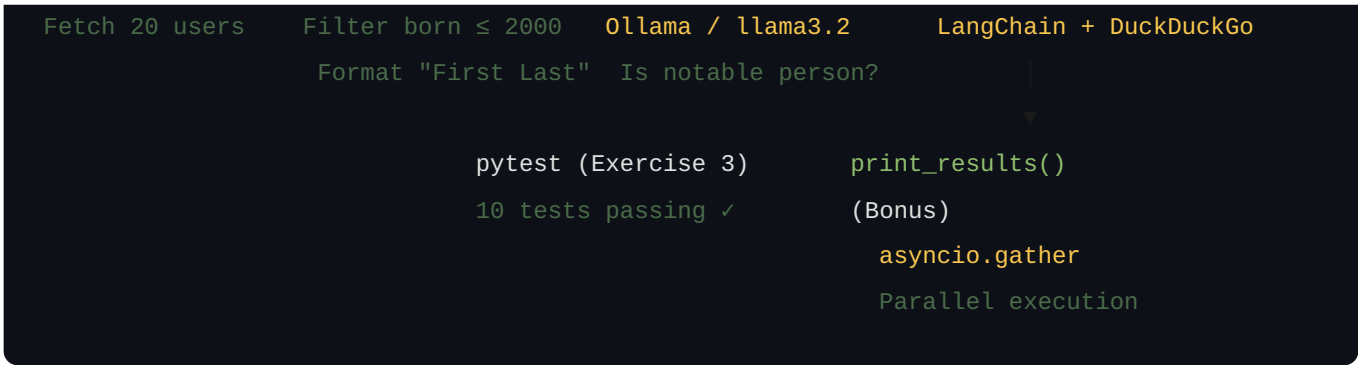
Better result presentation

- ✓ print_results() with summary header
- ✓ Trophy for public figures, circle for private
- ✓ Count of public vs private individuals
- ✓ Safe .get() dict access with defaults

Performance improvements

- ✓ asyncio.gather — all calls in parallel
- ✓ get_running_loop() — Python 3.10+ safe
- ✓ lru_cache(maxsize=256) for repeated names
- ✓ tenacity retry — 2s → 4s → 8s back-off
- ✓ before_sleep_log — automatic retry logging
- ✓ structured logging replaces bare print()

Architecture flow



FEEDBACK

💬 Assessment feedback

DIFFICULTY	TIME TAKEN	ENJOYMENT
Medium	~90 min	High
Well-calibrated for a GenAI Engineer role	Exercises 1–5 within window; bonus adds ~20 min	Logical progression from API to agentic AI

What went well

- ✓ randomuser.me is simple and well-documented
- ✓ create_react_agent makes the loop concise
- ✓ tenacity and lru_cache are drop-in improvements
- ✓ Clear incremental exercise structure

Observations & suggestions

- ✓ randomuser.me generates fictional people — LLM always "not famous" (expected)
- ✓ DuckDuckGo rate limits require careful batching
- ✓ llama3.2 hallucinates — gpt-4o-mini recommended for production
- ✓ asyncio.gather cuts wall time from n×delay to ~1 call duration

Tech stack

- Python 3.12
- requests
- pytest
- Ollama + llama3.2
- LangChain + LangGraph
- DuckDuckGo
- tenacity
- asyncio